Дисципліна «Технології DevOps»

ЗВІТ З ЛАБОРАТОРНОЇ РОБОТИ №3

Тема: «Розгортання власного застосунку в Kubernetes»

Студент	Короленко Влад Романович (vlkorolenko)
Репозиторій проєкту	https://github.com/vlkorolenko/devops-lab1
Стек застосунку	DevOps TechStore: Spring Boot (product-service, order-service), PostgreSQL 16, Nginx SPA
Середовище Kubernetes	Minikube v1.39.0 (driver=docker), Kubernetes v1.37.0, Namespace: devops-techstore

1. Підготувати образ застосунку

1а) Використати образ власного проєкту, зібраний у попередніх лабораторних

Використано власні Dockerfiles мікросервісного магазину DevOps TechStore (product-service, order-service, frontend) з Лабораторних №1–2. У кожен Dockerfile додано аргумент збірки APP_VERSION для версіонування без використання latest.

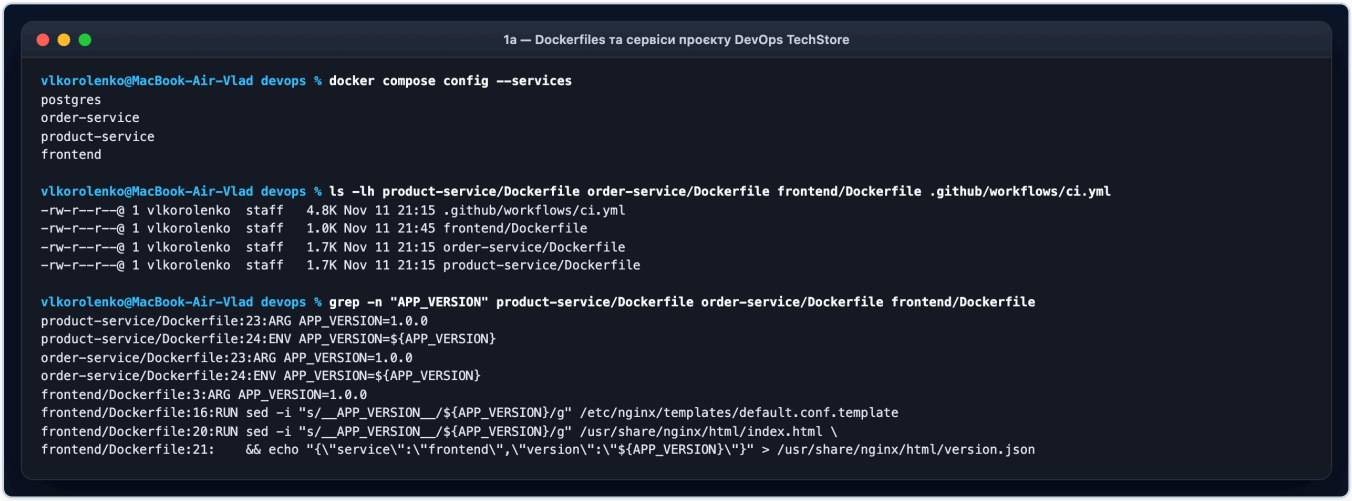


Рис. 1а — Сервіси Docker Compose, файли Dockerfile та підтримка APP_VERSION.

1b) Образ опублікований у registry (GHCR) та завантажений у minikube

Образи автоматично збираються CI/CD пайплайном GitHub Actions (multi-arch: linux/amd64, linux/arm64), публікуються в GitHub Container Registry (ghcr.io/vlkorolenko/devops-lab1/*) та завантажені локально в Minikube.

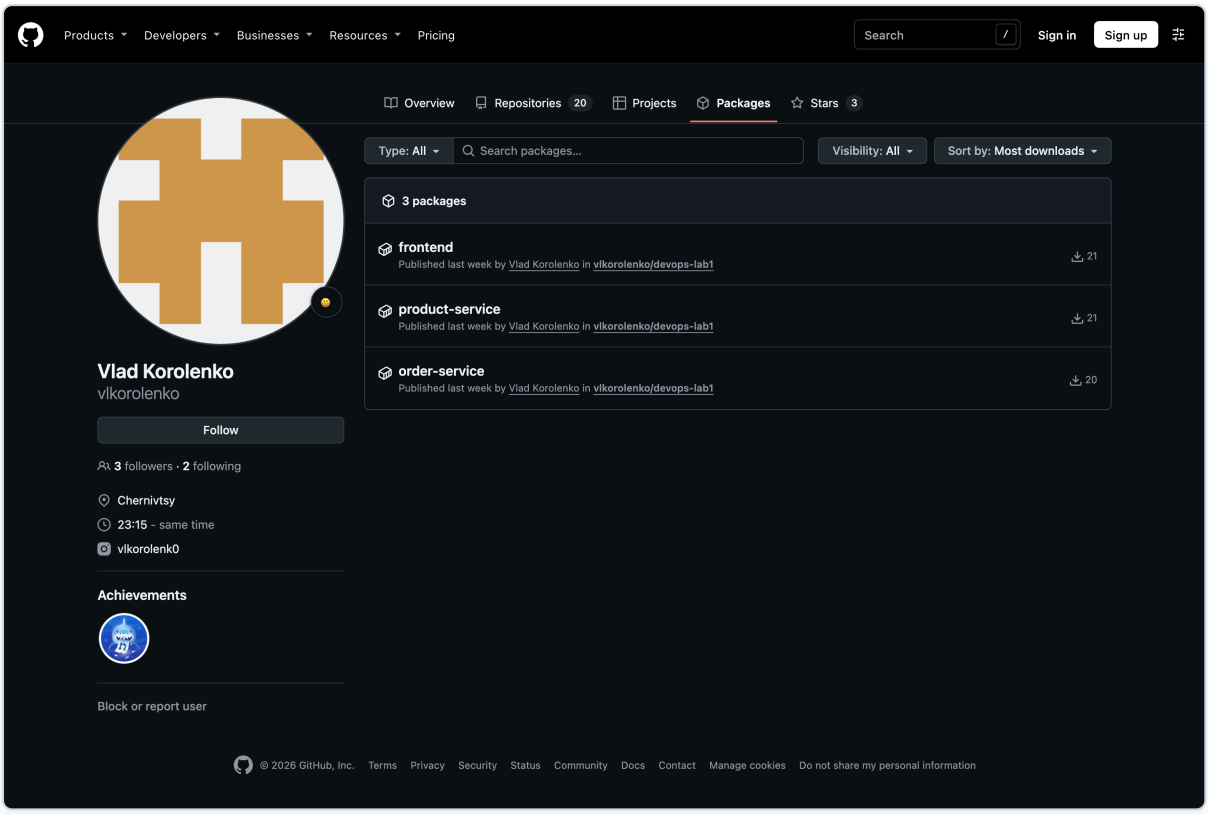


Рис. 1b.1 — Опубліковані контейнерні пакети frontend, product-service та order-service у GHCR.

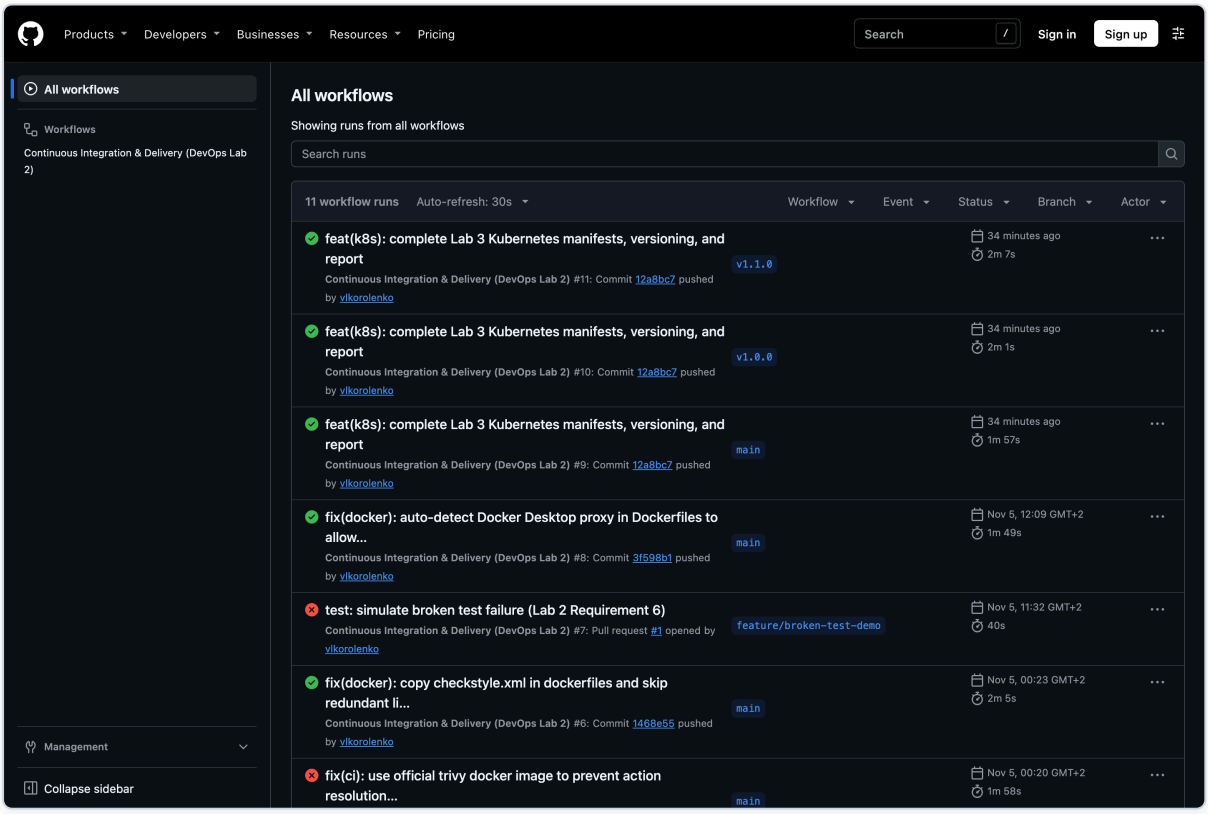


Рис. 1b.2 — Успішні збірки CI/CD у GitHub Actions для гілки main та релізних тегів v1.0.0 і v1.1.0.

1c) Образ позначити конкретним тегом версії, тег latest не використовувати

У всіх маніфестах Kubernetes вказані фіксовані теги версій (:1.0.0 та postgres:16-alpine). Пошук по каталогу k8s/ підтверджує повну відсутність тегу :latest.

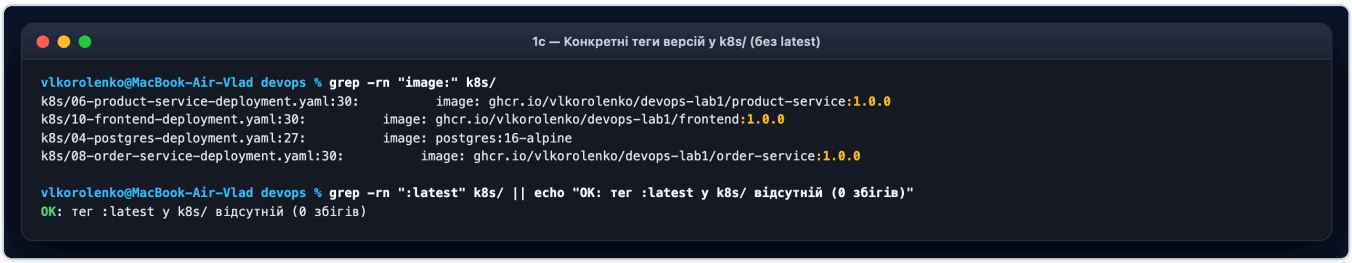


Рис. 1c — Перевірка фіксованих тегів образів і відсутності :latest у маніфестах k8s/.

1d) Підготувати щонайменше дві версії образу (1.0.0 і 1.1.0)

Підготовлено та завантажено у Minikube й GHCR дві версії образів — 1.0.0 та 1.1.0 — для демонстрації плавного оновлення (Rolling Update) та відкату (Rollback).



Рис. 1d — Git-теги v1.0.0, v1.1.0 та список образів обох версій у кластері Minikube.

2. Розгорнути локальний кластер

2a) Встановити minikube та kubectl

Встановлено клієнтські утиліти minikube v1.39.0 та kubectl v1.37.2 (через Homebrew у /opt/homebrew/bin).



Рис. 2a — Перевірка встановлених версій minikube та kubectl.

2b) Запустити кластер з драйвером docker

Локальний однонодовий кластер Kubernetes v1.37.0 запущено з драйвером docker (рантайм containerd, адреса ноди 192.168.49.2).

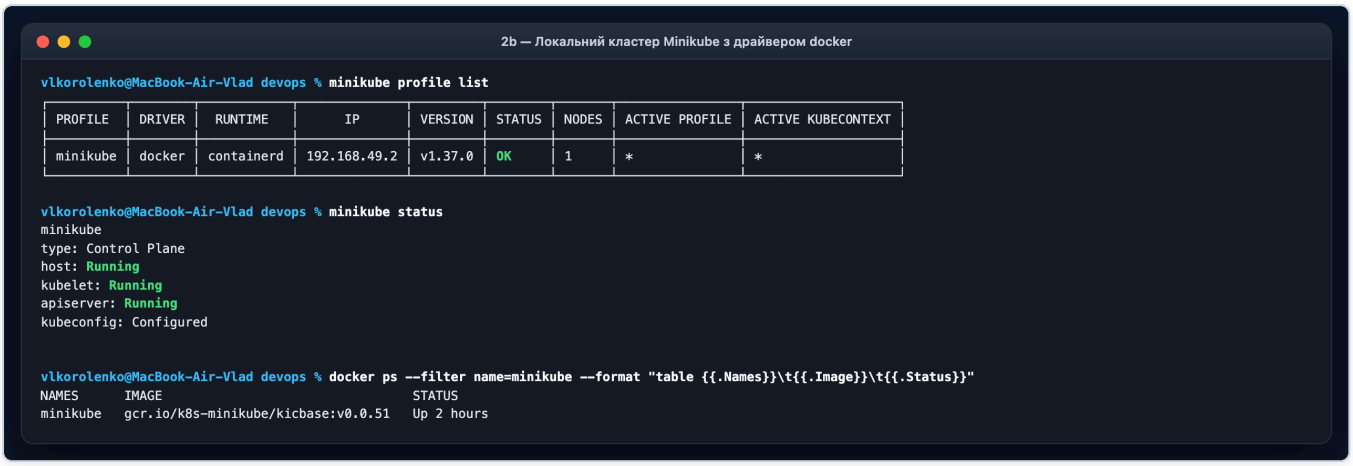


Рис. 2b — Стан профілю minikube з драйвером docker та запущений контейнер кластера.

2c) Перевірити стан вузла та системних подів у namespace kube-system

Вузол minikube перебуває у стані Ready, усі базові системні компоненти (coredns, etcd, kube-apiserver, kube-proxy, kube-scheduler, metrics-server, storage-provisioner) мають статус Running.

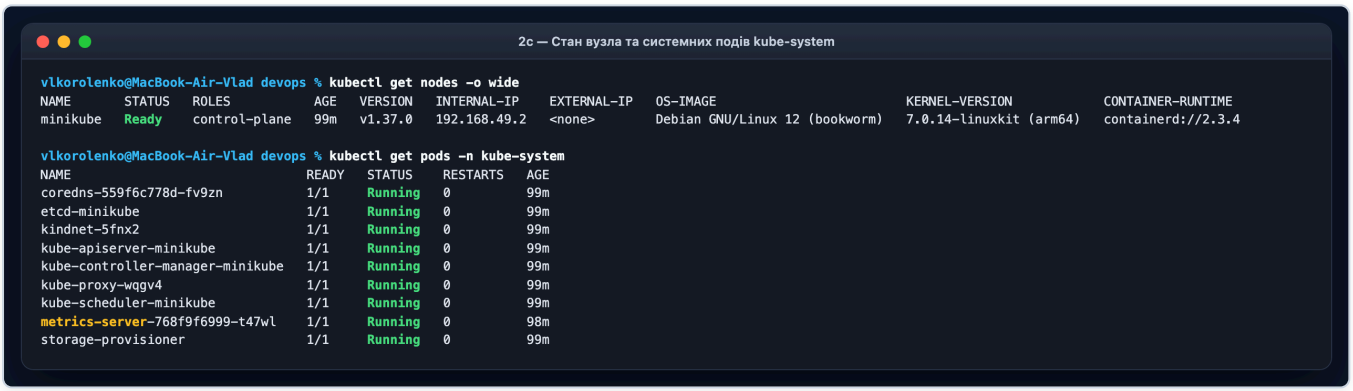


Рис. 2c — Перевірка стану вузла (kubectl get nodes) та системних подів kube-system.

2d) Увімкнути доповнення metrics-server та ingress

Увімкнено обидва необхідні аддони Minikube (metrics-server та ingress). Поди ingress-nginx запущені, а команда kubectl top nodes коректно повертає споживання CPU і RAM.



Рис. 2d — Список увімкнених аддонів, поди ingress-nginx та метрики kubectl top nodes.

3. Створити окремий namespace для проєкту

3a) Namespace назвати за іменем проєкту або прізвищем студента

Створено маніфест k8s/00-namespace.yaml, який визначає ізольований простір імен devops-techstore (статус Active).

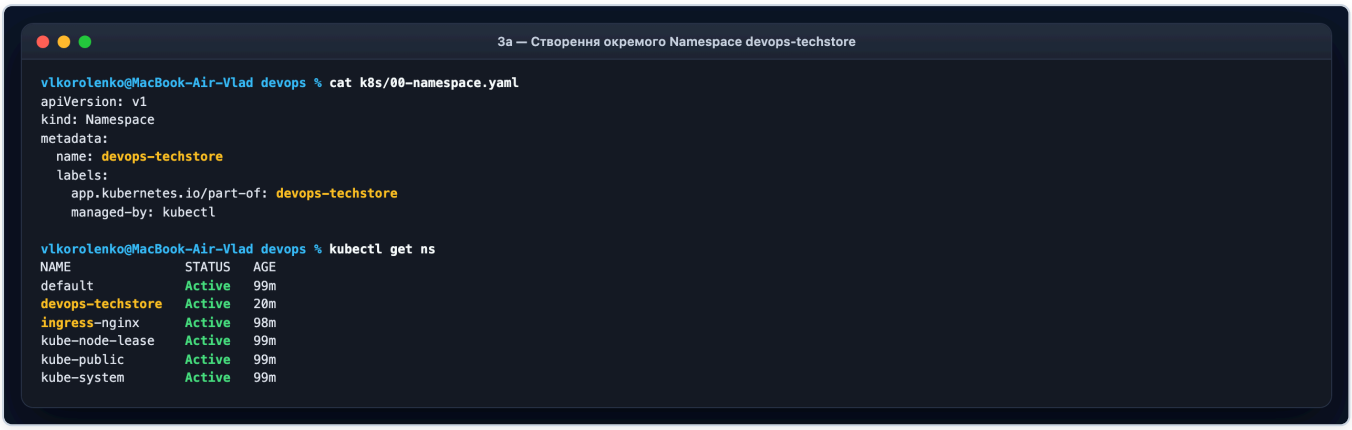


Рис. 3a — Маніфест k8s/00-namespace.yaml та перелік просторів імен (devops-techstore Active).

3b) Усі подальші об'єкти створювати лише в цьому namespace

У кожному з маніфестів k8s/01..12 явно зазначено metadata.namespace: devops-techstore, що забезпечує повну ізоляцію всіх ресурсів проєкту.



Рис. 3b — Підтвердження namespace: devops-techstore в усіх YAML-файлах і перелік об'єктів у кластері.

4. Описати маніфести застосунку

4a) Усі маніфести зберігати в каталозі k8s/, по одному файлу на об'єкт

Каталог k8s/ містить 13 написаних вручну YAML-файлів, структурованих суворо за правилом «один файл — один об'єкт Kubernetes (kind)».

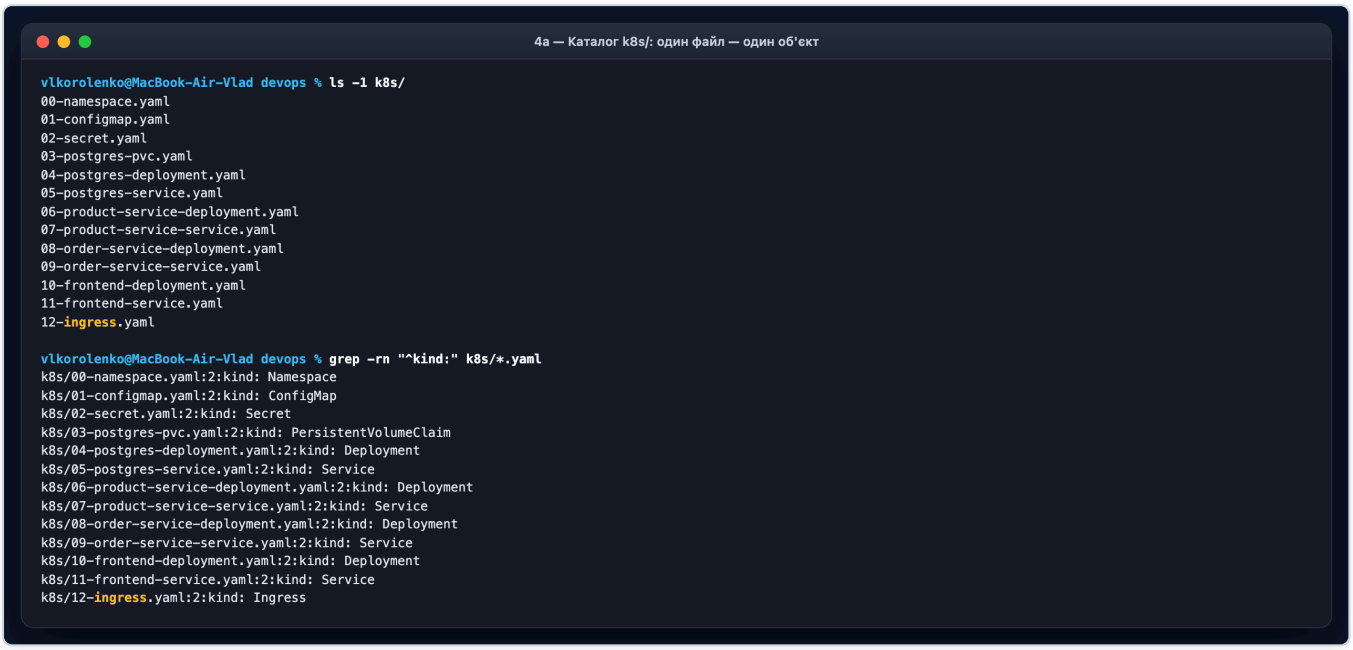


Рис. 4a — Вміст каталогу k8s/ та відповідність «1 файл = 1 об'єкт».

4b) Deployment: ≥ 2 репліки, labels/selector, requests/limits, liveness/readiness-проби

Застосункові сервіси (product-service, order-service, frontend) мають replicas: 2, узгоджені matchLabels і template.labels, обмеження CPU/RAM (requests і limits) та HTTP readinessProbe / livenessProbe.

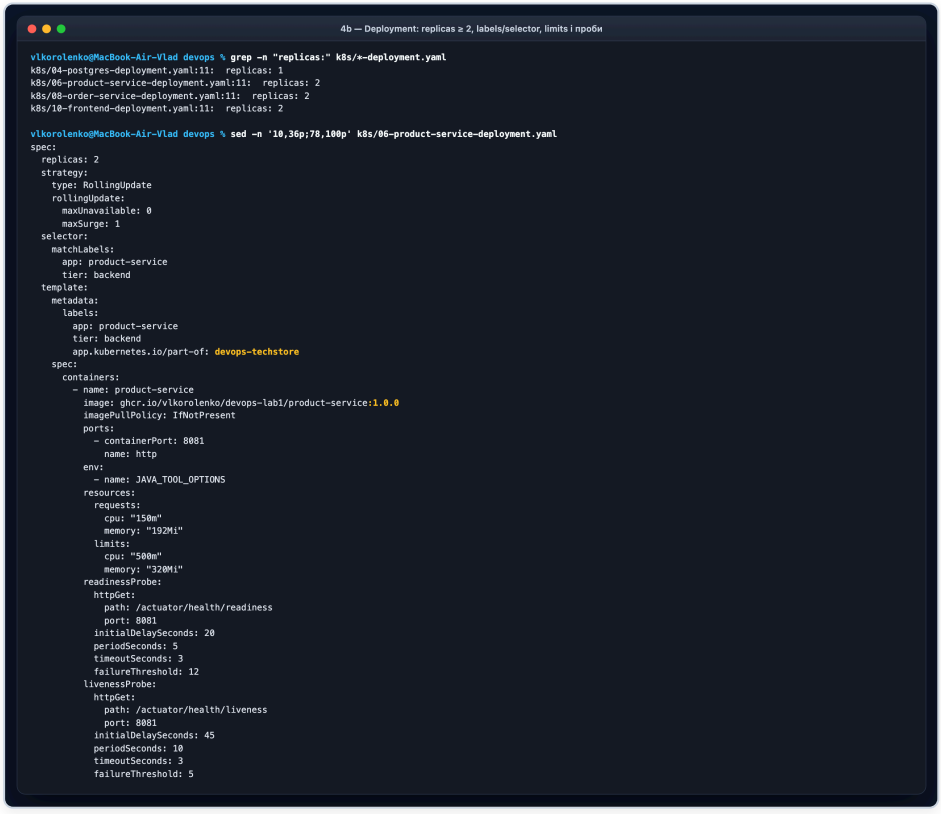


Рис. 4b — Кількість реплік, selector/labels, requests/limits та проби у Deployment маніфестах.

4c) Service: доступ всередині кластера та з хоста

Для внутрішніх сервісів (postgres, product-service, order-service) налаштовано ClusterIP, а для frontend — NodePort (порт 80 -> 30080) для зовнішнього доступу.

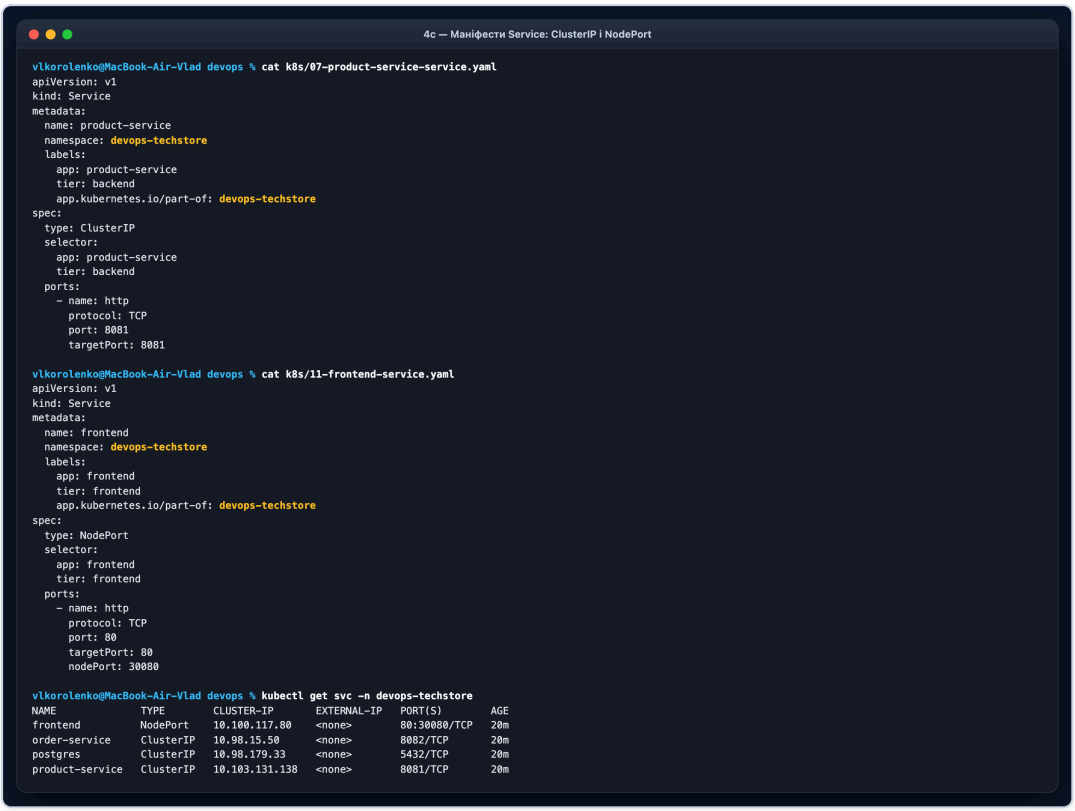


Рис. 4c — Маніфести ClusterIP і NodePort сервісів та їх перелік у кластері.

4d) ConfigMap: не менше двох параметрів конфігурації (env або файл)

Маніфест k8s/01-configmap.yaml (techstore-config) передає 7 змінних оточення (через configMapKeyRef) і JSON-файл store-config.json (змонтований у контейнер frontend через volumeMounts).

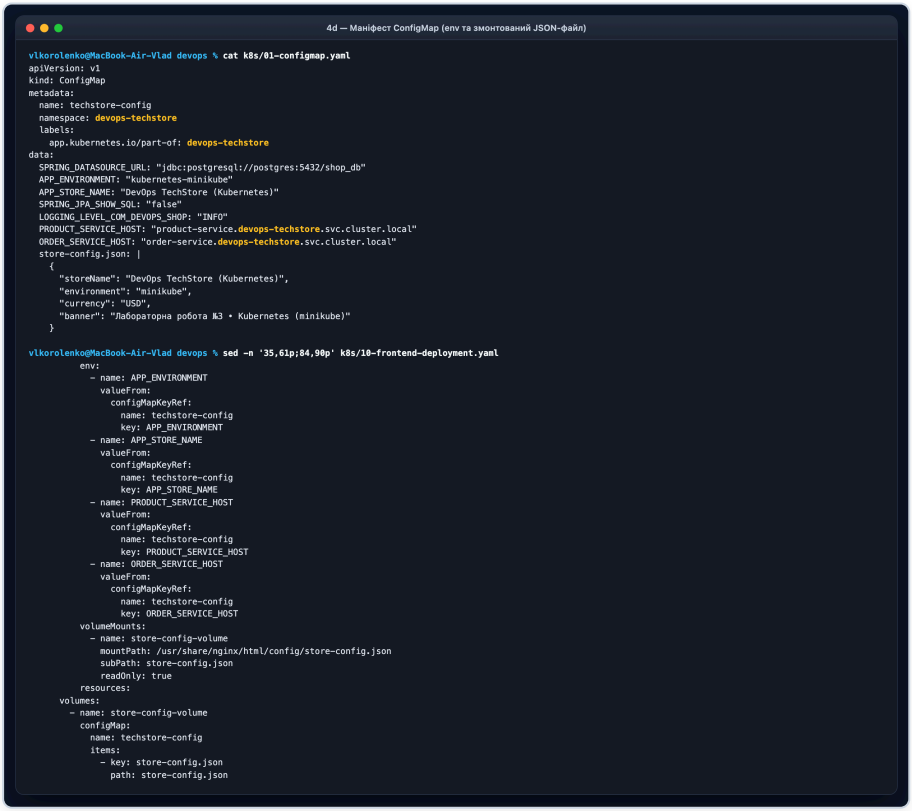


Рис. 4d — Вміст ConfigMap techstore-config та підключення env і файлу у frontend Deployment.

4e) Secret: щонайменше один чутливий параметр (тестові дані)

У k8s/02-secret.yaml (techstore-secret) описано тестові облікові дані PostgreSQL та сервісний ключ INTERNAL_API_KEY, підключені через secretKeyRef (реальні паролі не зберігаються).

4e — Маніфест Secret (тестові дані підключення та ключ)

```
vlkorolenko@MacBook-Air-Vlad devops % cat k8s/02-secret.yaml
apiVersion: v1
kind: Secret
metadata:
  name: techstore-secret
  namespace: devops-techstore
  labels:
    app.kubernetes.io/part-of: devops-techstore
type: Opaque
stringData:
  POSTGRES_DB: "shop_db"
  POSTGRES_USER: "shop_user"
  POSTGRES_PASSWORD: "k8s-demo-db-password-2026"
  SPRING_DATASOURCE_USERNAME: "shop_user"
  SPRING_DATASOURCE_PASSWORD: "k8s-demo-db-password-2026"
  INTERNAL_API_KEY: "k8s-demo-internal-api-key"

vlkorolenko@MacBook-Air-Vlad devops % sed -n '63,77p' k8s/06-product-service-deployment.yaml
            - name: SPRING_DATASOURCE_USERNAME
              valueFrom:
                secretKeyRef:
                  name: techstore-secret
                  key: SPRING_DATASOURCE_USERNAME
            - name: SPRING_DATASOURCE_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: techstore-secret
                  key: SPRING_DATASOURCE_PASSWORD
            - name: INTERNAL_API_KEY
              valueFrom:
                secretKeyRef:
                  name: techstore-secret
                  key: INTERNAL_API_KEY
```

Рис. 4e — Опис Secret techstore-secret та підключення секретних ключів у Deployment.

4f) Ingress для маршрутизації зовнішнього HTTP-трафіку

У k8s/12-ingress.yaml описано об'єкт Ingress (ingressClassName: nginx), який маршрутизує вхідні HTTP-запити (/) на сервіс frontend:80.

4f — Маніфест Ingress (ingress-nginx -> frontend)

```
vlkorolenko@MacBook-Air-Vlad devops % cat k8s/12-ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: techstore-ingress
  namespace: devops-techstore
  labels:
    app.kubernetes.io/part-of: devops-techstore
spec:
  ingressClassName: nginx
  rules:
    - http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend
                port:
                  number: 80

vlkorolenko@MacBook-Air-Vlad devops % kubectl get ingress techstore-ingress -n devops-techstore
NAME               CLASS   HOSTS   ADDRESS        PORTS   AGE
techstore-ingress   nginx   *       192.168.49.2   80      20m
```

Рис. 4f — Маніфест k8s/12-ingress.yaml та активна адреса Ingress контролера.

4g) PersistentVolumeClaim для бази даних PostgreSQL

Для компоненту зі станом (PostgreSQL) у k8s/03-postgres-pvc.yaml створено PVC postgres-pvc обсягом 1Gi (ReadWriteOnce), змонтований у каталог /var/lib/postgresql/data.

4g — Маніфест PersistentVolumeClaim для PostgreSQL

```
vlkorolenko@MacBook-Air-Vlad devops % cat k8s/03-postgres-pvc.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: postgres-pvc
  namespace: devops-techstore
  labels:
    app: postgres
    app.kubernetes.io/part-of: devops-techstore
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi

vlkorolenko@MacBook-Air-Vlad devops % sed -n '48,51p;79,82p' k8s/04-postgres-deployment.yaml
          volumeMounts:
            - name: postgres-storage
              mountPath: /var/lib/postgresql/data
              subPath: pgdata
      volumes:
        - name: postgres-storage
          persistentVolumeClaim:
            claimName: postgres-pvc

vlkorolenko@MacBook-Air-Vlad devops % kubectl get pvc -n devops-techstore
NAME           STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
postgres-pvc   Bound    pvc-76b7db5c-8e83-4280-b97a-c4eba4c505e3   1Gi        RWO            standard       <unset>                 20m
```

Рис. 4g — Маніфест postgres-pvc, підключення тому в Deployment та статус Bound.

5. Розгорнути застосунок

5a) Застосувати всі маніфести однією командою з каталогу

Весь стек застосунку розгорнуто з нуля однією командою kubectl apply -f k8s/ (усі 13 об'єктів створені автоматично, Deployment-и перейшли у стан Available).

5a — Розгортання всіх маніфестів однією командою

```
vlkorolenko@MacBook-Air-Vlad devops % kubectl apply -f k8s/
namespace/devops-techstore created
configmap/techstore-config created
secret/techstore-secret created
persistentvolumeclaim/postgres-pvc created
deployment.apps/postgres created
service/postgres created
deployment.apps/product-service created
service/product-service created
deployment.apps/order-service created
service/order-service created
deployment.apps/frontend created
service/frontend created
ingress.networking.k8s.io/techstore-ingress created

vlkorolenko@MacBook-Air-Vlad devops % kubectl wait --for=condition=available deployment --all -n devops-techstore --timeout=180s
deployment.apps/frontend condition met
deployment.apps/order-service condition met
deployment.apps/postgres condition met
deployment.apps/product-service condition met
```

Рис. 5a — Розгортання всього стеку однією командою kubectl apply -f k8s/.

5b) Показати перелік створених об'єктів: поди, Deployment, ReplicaSet, Service

Усі 7 подів (2 frontend, 2 product-service, 2 order-service, 1 postgres), 4 Deployments, 4 ReplicaSets, 4 Services, Ingress і PVC працюють штатно; kubectl top pods показує актуальні метрики.

5b — Створені поди, Deployment, ReplicaSet, Service та метрики

```
vlkorolenko@MacBook-Air-Vlad devops % kubectl get pods,deployments,replicasets,services,ingress,pvc -n devops-techstore -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP            NODE
pod/frontend-5d88869466-gkk4b       1/1     Running   0          73s   10.244.0.21   minikube
pod/frontend-5d88869466-hgm7f       1/1     Running   0          73s   10.244.0.22   minikube
pod/order-service-5499bc87c7-2p22q  1/1     Running   0          73s   10.244.0.20   minikube
pod/order-service-5499bc87c7-7zpfk  1/1     Running   0          73s   10.244.0.19   minikube
pod/postgres-6d6dc64d7f-5npd5       1/1     Running   0          73s   10.244.0.18   minikube
pod/product-service-545fb94449-jxpr5  1/1   Running   0          73s   10.244.0.16   minikube
pod/product-service-545fb94449-kfntg  1/1   Running   0          73s   10.244.0.17   minikube

NAME                              READY   UP-TO-DATE   AVAILABLE   AGE   IMAGES
deployment.apps/frontend          2/2     2            2           73s   ghcr.io/vlkorolenko/devops-lab1/frontend:1.0.0
deployment.apps/order-service     2/2     2            2           73s   ghcr.io/vlkorolenko/devops-lab1/order-service:1.0.0
deployment.apps/postgres          1/1     1            1           73s   postgres:16-alpine
deployment.apps/product-service   2/2     2            2           73s   ghcr.io/vlkorolenko/devops-lab1/product-service:1.0.0

NAME                                        DESIRED   CURRENT   READY   AGE
replicaset.apps/frontend-5d88869466         2         2         2       73s
replicaset.apps/order-service-5499bc87c7    2         2         2       73s
replicaset.apps/postgres-6d6dc64d7f         1         1         1       73s
replicaset.apps/product-service-545fb94449  2         2         2       73s

NAME                      TYPE        CLUSTER-IP       PORT(S)        SELECTOR
service/frontend          NodePort    10.111.202.233   80:30080/TCP   app=frontend,tier=frontend
service/order-service     ClusterIP   10.104.219.111   8082/TCP       app=order-service,tier=backend
service/postgres          ClusterIP   10.99.219.241    5432/TCP       app=postgres,tier=database
service/product-service   ClusterIP   10.96.79.141     8081/TCP       app=product-service,tier=backend

vlkorolenko@MacBook-Air-Vlad devops % kubectl top pods -n devops-techstore
NAME                               CPU(cores)   MEMORY(bytes)
frontend-5d88869466-gkk4b          10m          7Mi
frontend-5d88869466-hgm7f          8m           7Mi
order-service-5499bc87c7-2p22q     494m         136Mi
order-service-5499bc87c7-7zpfk     493m         167Mi
postgres-6d6dc64d7f-5npd5          131m         57Mi
product-service-545fb94449-jxpr5   491m         122Mi
product-service-545fb94449-kfntg   494m         127Mi
```

Рис. 5b — Перелік Pod, Deployment, ReplicaSet, Service у devops-techstore та метрики top pods.

5c) Відкрити застосунок у браузері та показати, що він працює

У Google Chrome через Ingress контролер відкрито DevOps TechStore (v1.0.0): відображається каталог 6 товарів із PostgreSQL, банер із ConfigMap, ім'я пода та статус «Всі сервіси UP». Створене замовлення #1 зберігається в PostgreSQL і відображається у вікні «Мої замовлення».

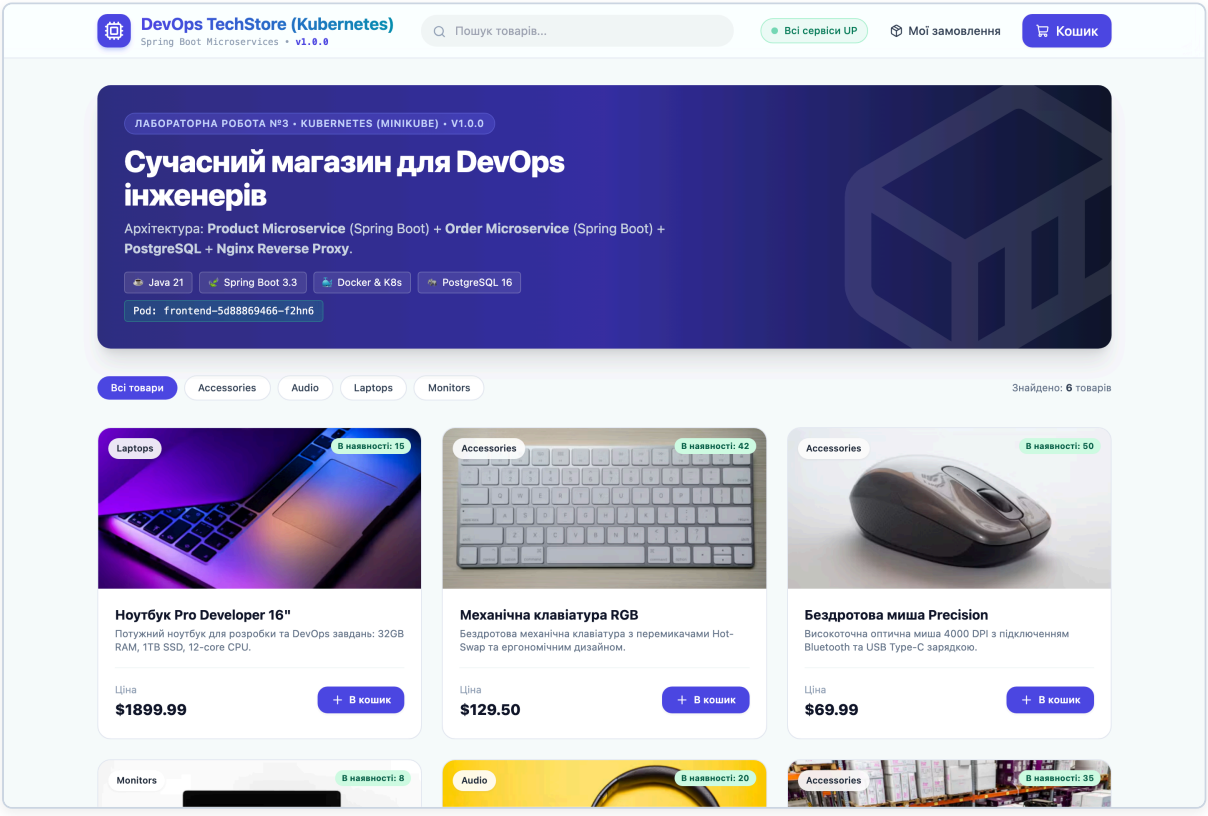


Рис. 5c.1 — Вебінтерфейс DevOps TechStore v1.0.0 у браузері з товарами із PostgreSQL.

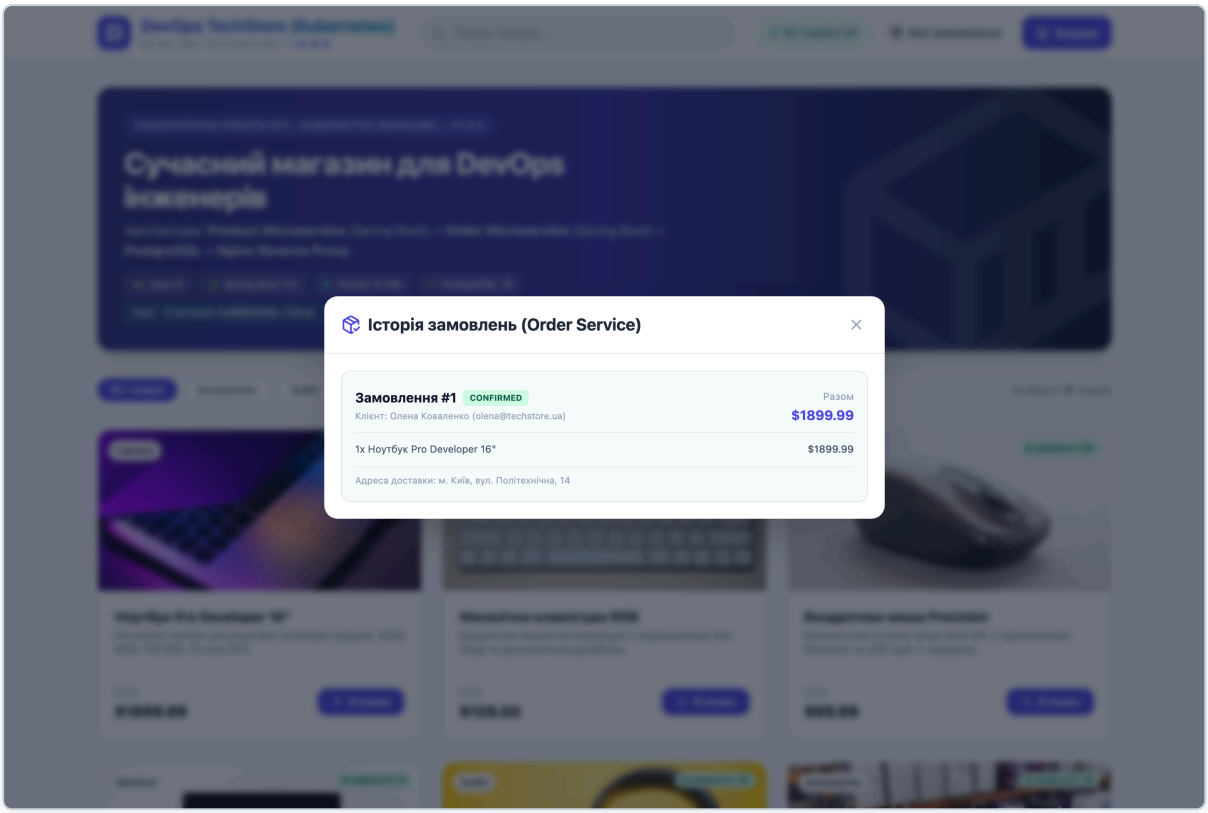


Рис. 5c.2 — Модальне вікно «Історія замовлень (Order Service)»: замовлення #1 у PostgreSQL.

5d) Показати, що значення з ConfigMap і Secret справді потрапили в контейнер

Через kubectl exec перевірено змінні оточення всередині product-service і frontend, вміст змонтованого файлу /usr/share/nginx/html/config/store-config.json та відповідь /actuator/product-service/info.

5d — Потрапляння значень ConfigMap і Secret усередину контейнерів

```
vlkorolenko@MacBook-Air-Vlad devops % kubectl get configmap,secret -n devops-techstore
NAME                         DATA   AGE
configmap/kube-root-ca.crt   1      73s
configmap/techstore-config   8      73s

NAME                     TYPE     DATA   AGE
secret/techstore-secret  Opaque   6      73s

vlkorolenko@MacBook-Air-Vlad devops % kubectl exec -n devops-techstore deploy/product-service -- env | grep -E '^(SPRING_DATASOURCE_|APP_|INTERNAL_API_KEY)' | sort
APP_ENVIRONMENT=kubernetes-minikube
APP_STORE_NAME=DevOps TechStore (Kubernetes)
APP_VERSION=1.0.0
INTERNAL_API_KEY=k8s-demo-internal-api-key
SPRING_DATASOURCE_PASSWORD=k8s-demo-db-password-2026
SPRING_DATASOURCE_URL=jdbc:postgresql://postgres:5432/shop_db
SPRING_DATASOURCE_USERNAME=shop_user

vlkorolenko@MacBook-Air-Vlad devops % kubectl exec -n devops-techstore deploy/frontend -- env | grep -E '^(APP_|PRODUCT_SERVICE_HOST|ORDER_SERVICE_HOST)' | sort
APP_ENVIRONMENT=kubernetes-minikube
APP_STORE_NAME=DevOps TechStore (Kubernetes)
ORDER_SERVICE_HOST=order-service.devops-techstore.svc.cluster.local
PRODUCT_SERVICE_HOST=product-service.devops-techstore.svc.cluster.local

vlkorolenko@MacBook-Air-Vlad devops % kubectl exec -n devops-techstore deploy/frontend -- cat /usr/share/nginx/html/config/store-config.json
{
  "storeName": "DevOps TechStore (Kubernetes)",
  "environment": "minikube",
  "currency": "USD",
  "banner": "Лабораторна робота №3 • Kubernetes (minikube)"
}

vlkorolenko@MacBook-Air-Vlad devops % curl -s http://127.0.0.1:18090/actuator/product-service/info
{"app":{"name":"product-service","version":"1.0.0","environment":"kubernetes-minikube","storeName":"DevOps TechStore (Kubernetes)"}}
```

Рис. 5d — Перевірка змінних оточення ConfigMap/Secret, файлу store-config.json та /actuator/product-service/info.

6. Продемонструвати керування життєвим циклом

6a) Самовідновлення: видалити под і показати, що кластер створив новий

Після видалення пода frontend-5d88869466-gkk4b командою kubectl delete pod контролер ReplicaSet автоматично створив новий под (frontend-5d88869466-zt46s) для підтримання бажаних 2 реплік.

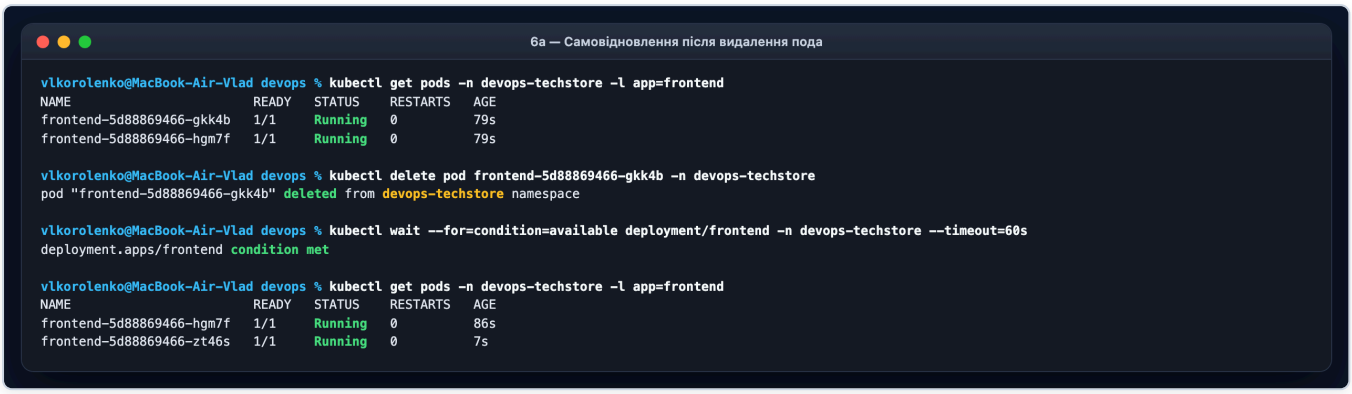


Рис. 6a — Демонстрація самовідновлення (Self-Healing) після видалення пода frontend.

6b) Масштабування: збільшити кількість реплік і показати розподіл трафіку

Кількість реплік frontend збільшено з 2 до 4 (kubectl scale --replicas=4). Послідовні 20 HTTP-запитів через NodePort (30080) та Ingress (80) розподілилися між усіма 4 подами.

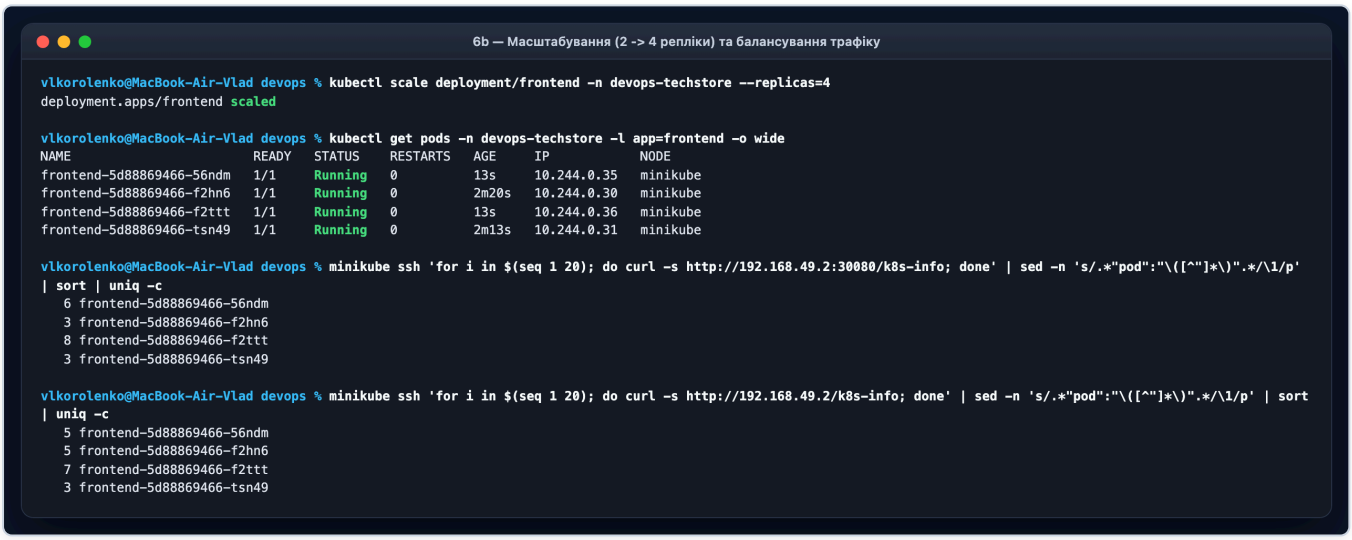


Рис. 6b — Масштабування до 4 реплік та розподіл 20 запитів через NodePort і Ingress.

6c) Оновлення: замінити образ на другу версію (1.1.0) та показати rolling update

За допомогою kubectl set image образи frontend та product-service оновлено з 1.0.0 до 1.1.0 зі стратегією RollingUpdate. Нова версія v1.1.0 підтверджена як в API, так і в браузері.

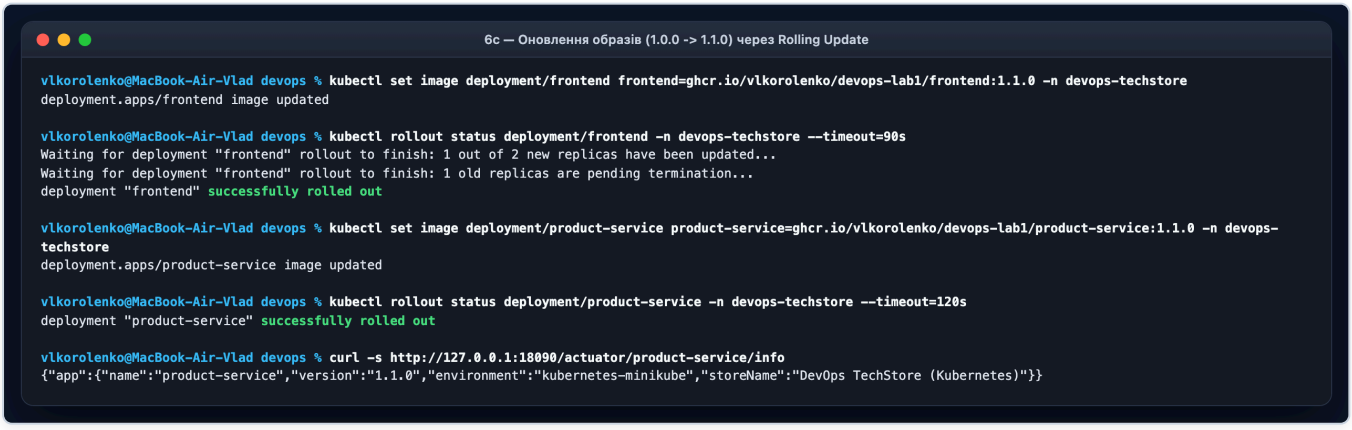


Рис. 6c.1 — Оновлення frontend і product-service до версії 1.1.0 через Rolling Update.

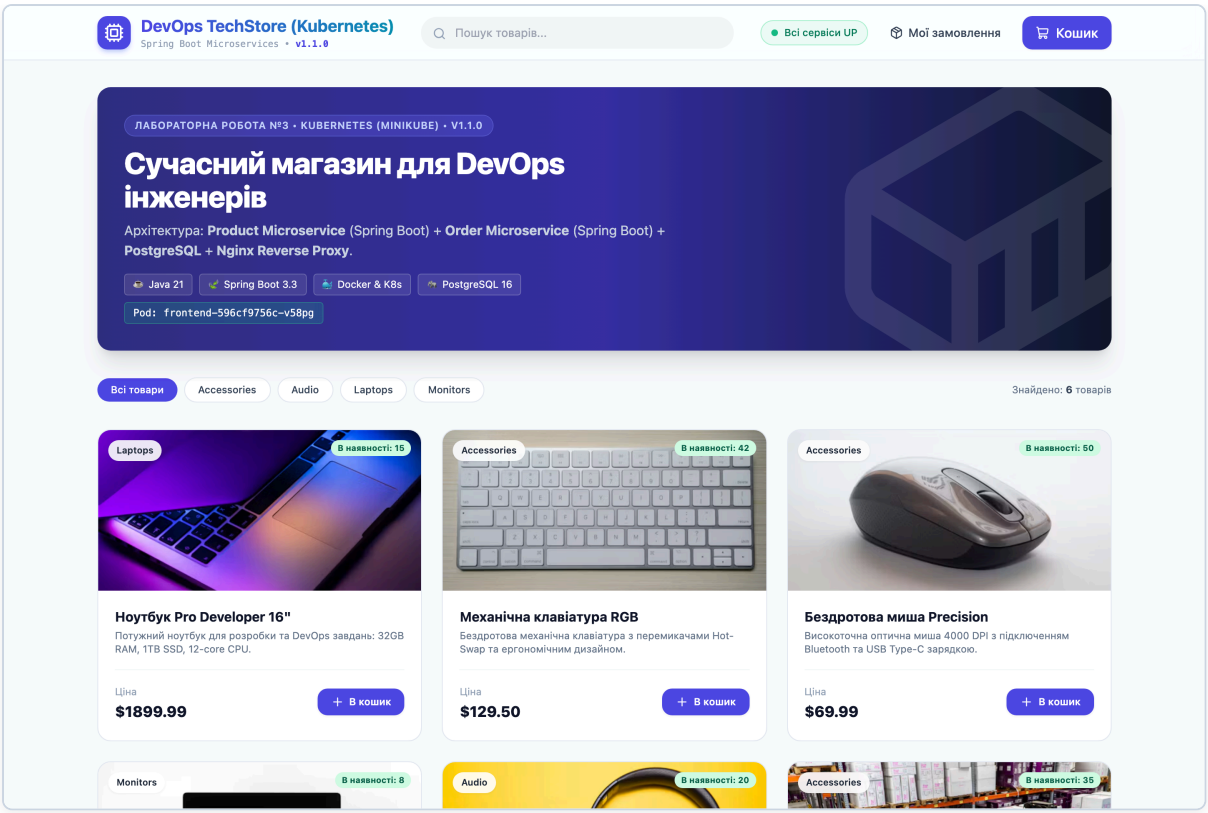


Рис. 6c.2 — Вітрина DevOps TechStore у браузері після переходу на версію v1.1.0.

6d) Історія оновлень та відкат до попередньої ревізії

Перевірено історію ревізій (kubectl rollout history) та виконано відкат до попередньої стабільної версії 1.0.0 командою kubectl rollout undo для обох сервісів.

6d — Історія ревізій та Rollback (1.1.0 -> 1.0.0)

```
vlkorolenko@MacBook-Air-Vlad devops % kubectl rollout history deployment/frontend -n devops-techstore
deployment.apps/frontend
REVISION  CHANGE-CAUSE
1         <none>
2         <none>

vlkorolenko@MacBook-Air-Vlad devops % kubectl rollout undo deployment/frontend -n devops-techstore
deployment.apps/frontend rolled back
deployment "frontend" successfully rolled out

vlkorolenko@MacBook-Air-Vlad devops % curl -s http://127.0.0.1:18090/k8s-info
{"service":"frontend","version":"1.0.0","pod":"frontend-5d88869466-f2hn6","status":"ok"}

vlkorolenko@MacBook-Air-Vlad devops % kubectl rollout undo deployment/product-service -n devops-techstore
deployment.apps/product-service rolled back
deployment "product-service" successfully rolled out

vlkorolenko@MacBook-Air-Vlad devops % curl -s http://127.0.0.1:18090/actuator/product-service/info
{"app":{"name":"product-service","version":"1.0.0","environment":"kubernetes-minikube","storeName":"DevOps TechStore (Kubernetes)"}}
```

Рис. 6d — Перегляд історії ревізій та Rollback (1.1.0 -> 1.0.0).

6e) Показати, що під час оновлення застосунок залишався доступним

Під час Rolling Update (1.0.0 -> 1.1.0) паралельно працював безперервний HTTP-зонд: зафіксовано 33 із 33 успішних відповідей HTTP 200 OK (Failures=0), жодного розриву обслуговування не було.

6e — Безперервна доступність застосунку під час Rolling Update

```
# Паралельне HTTP-зондування /k8s-info кожні 0.3 с під час оновлення 1.0.0 -> 1.1.0:
Rolling update probe summary:
Total requests=33, HTTP 200=33, Failures=0

19:51:39 HTTP=200 {"service":"frontend","version":"1.0.0","pod":"frontend-5d88869466-hgm7f","status":"ok"}
19:51:39 HTTP=200 {"service":"frontend","version":"1.0.0","pod":"frontend-5d88869466-zt46s","status":"ok"}
19:51:43 HTTP=200 {"service":"frontend","version":"1.0.0","pod":"frontend-5d88869466-hgm7f","status":"ok"}
19:51:47 HTTP=200 {"service":"frontend","version":"1.0.0","pod":"frontend-5d88869466-zt46s","status":"ok"}
19:51:52 HTTP=200 {"service":"frontend","version":"1.1.0","pod":"frontend-596cf9756c-tcnlw","status":"ok"}
19:51:52 HTTP=200 {"service":"frontend","version":"1.1.0","pod":"frontend-596cf9756c-76shj","status":"ok"}
19:51:53 HTTP=200 {"service":"frontend","version":"1.1.0","pod":"frontend-596cf9756c-tcnlw","status":"ok"}
```

Рис. 6e — Результати безперервного HTTP-зондування під час Rolling Update (33/33 HTTP 200 OK).

7. Виконати діагностику

7a) Навмисно вказати неіснуючий тег образу і показати стан ImagePullBackOff

Вказано неіснуючий тег frontend:9.9.9-nonexistent. Новий под перейшов у стан ImagePullBackOff, при цьому 2 попередні поди залишилися Running завдяки maxUnavailable: 0.

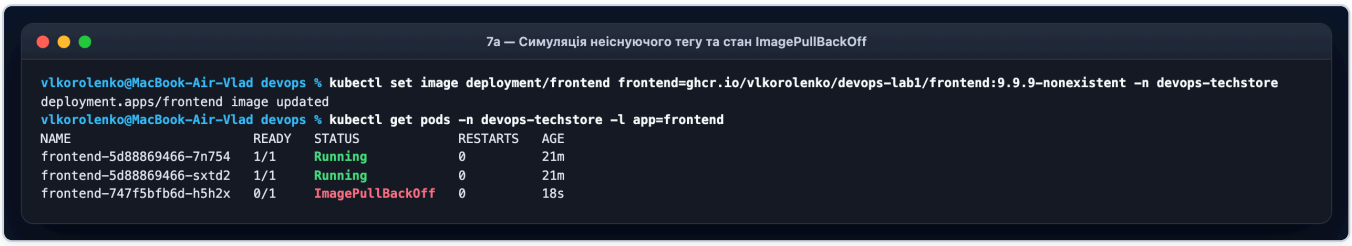


Рис. 7a — Помилка ImagePullBackOff при вказанні неіснуючого тегу 9.9.9-nonexistent.

7b) Знайти причину за допомогою опису пода та подій (Events)

У виводі kubectl describe pod в секції Events знайдено точну першопричину збою завантаження образу: failed to resolve reference ...: not found та ImagePullBackOff.

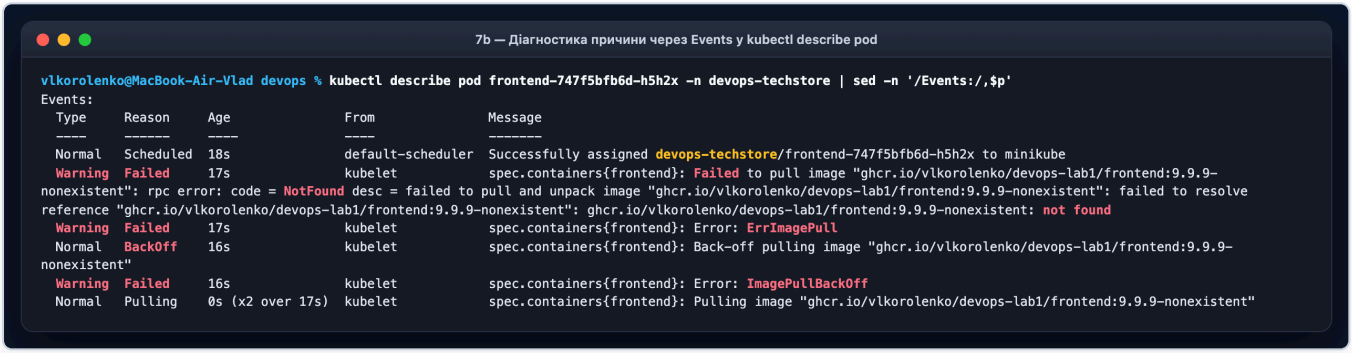


Рис. 7b — Діагностика причини помилки у секції Events через kubectl describe pod.

7c) Показати логи контейнера та вхід у контейнер в інтерактивному режимі

Отримано логи Spring Boot бекенду через kubectl logs та виконано інтерактивний вхід у контейнер frontend (kubectl exec -it ... -- sh) із перевіркою hostname і store-config.json.

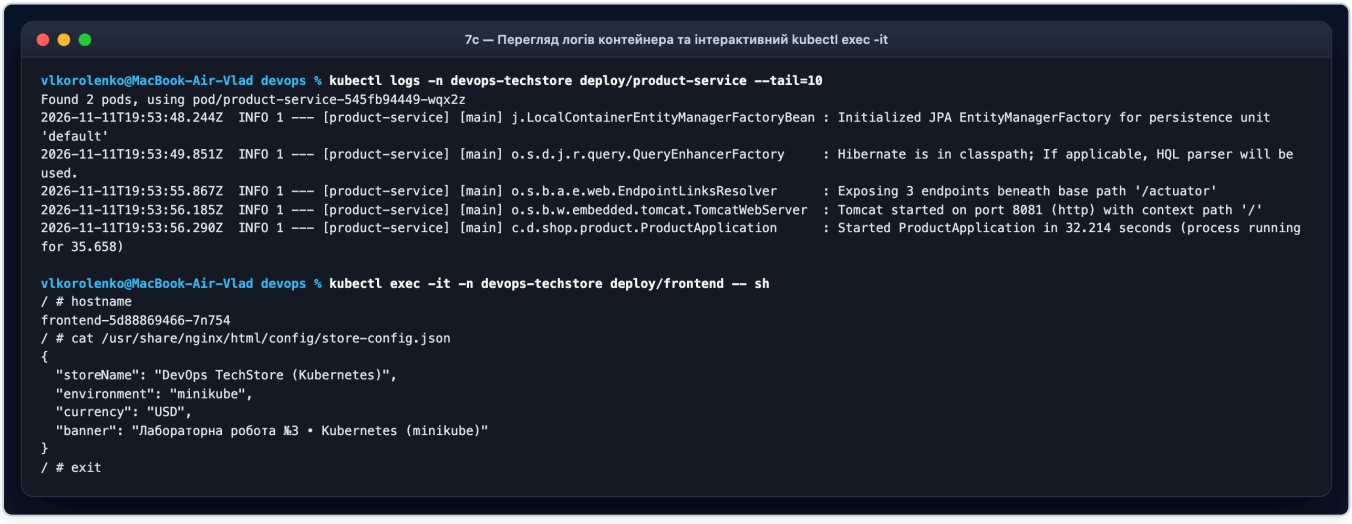


Рис. 7c — Перегляд логів (kubectl logs) та інтерактивна сесія kubectl exec -it.

7d) Виправити помилку і довести застосунок до робочого стану

Повернуто коректний тег образу frontend:1.0.0. Після завершення rollout усі 7 подів і 4 Deployments повністю відновилися у стані Running (RESTARTS: 0).

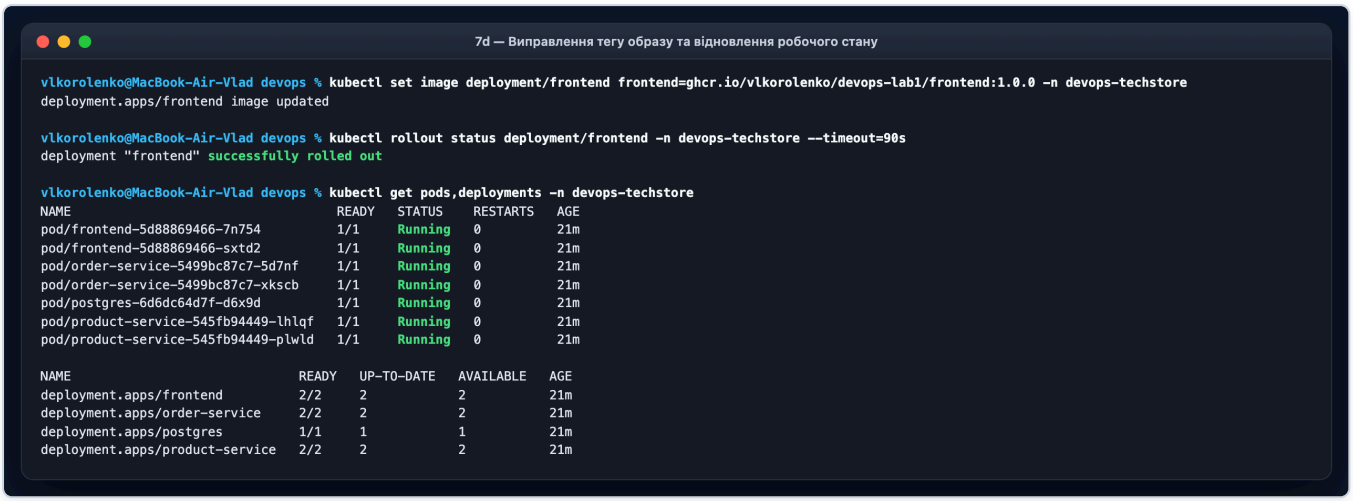


Рис. 7d — Виправлення тегу образу на 1.0.0 та підтвердження повного відновлення кластера.

8. Оформити результати

8a) Каталог k8s/ з маніфестами закомітити в репозиторій проєкту

Усі 13 маніфестів каталогу k8s/ та оновлені конфігурації закомічені й відправлені до GitHub-репозиторію vlkorolenko/devops-lab1 (гілка main).

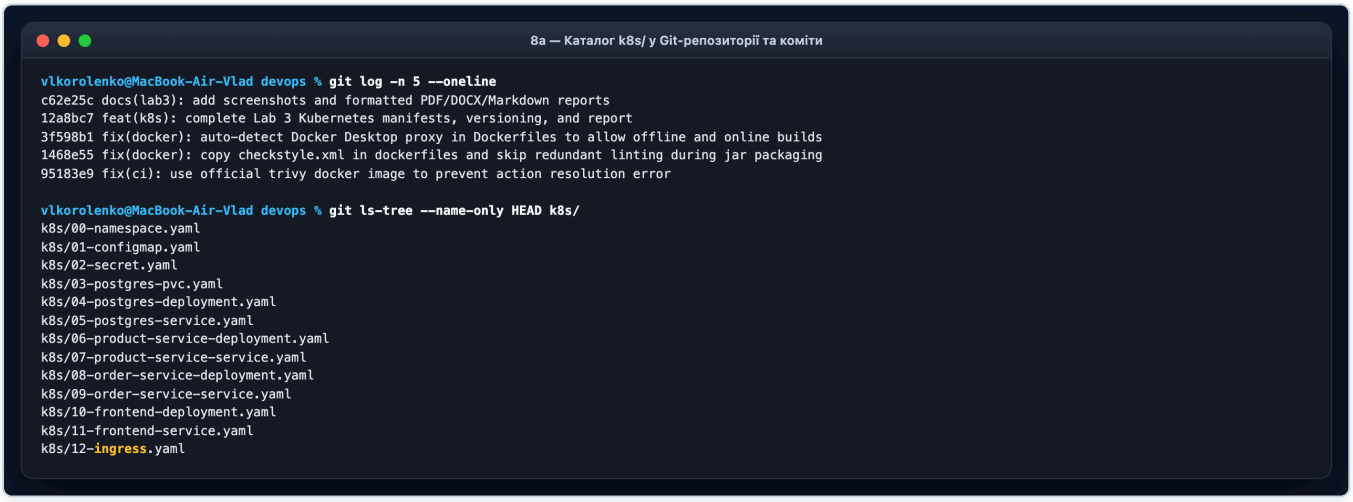


Рис. 8a — Історія комітів Git та список закомічених файлів у каталозі k8s/.

8b) У README описати порядок розгортання та перелік параметрів у ConfigMap і Secret

У файлі README.md детально задокументовано таблицю маніфестів k8s/, повний перелік ключів ConfigMap і Secret та покрокові команди розгортання кластера з нуля.

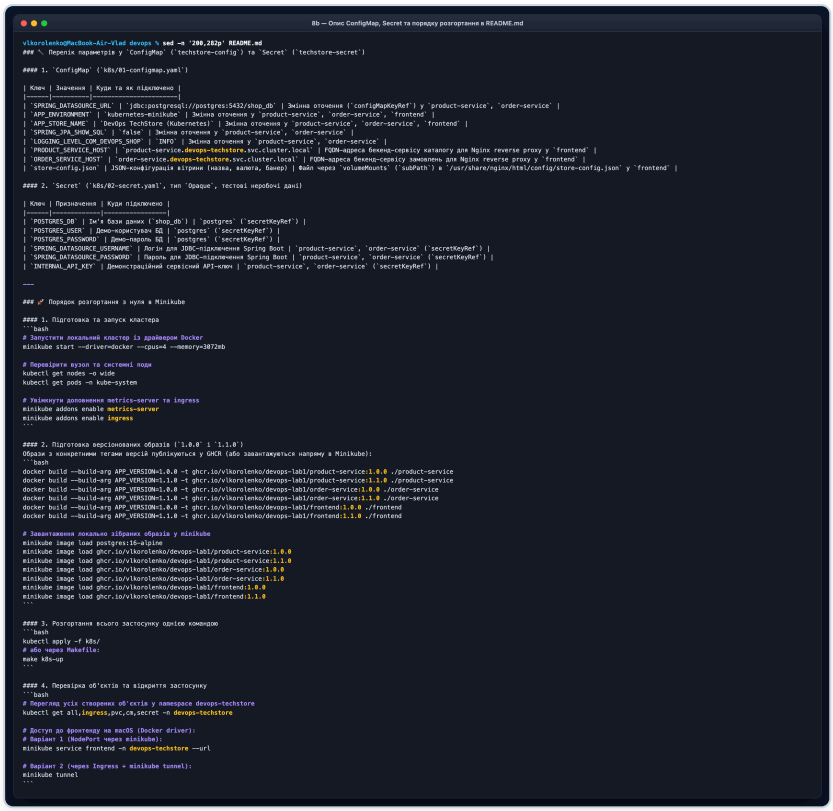


Рис. 8b — Розділ README.md із таблицями ConfigMap/Secret та інструкцією розгортання.

8c) Підготувати звіт зі скріншотами та виводом команд

Сформовано структурований звіт із покроковими скріншотами та збереженим виводом команд у форматах LAB3_REPORT.md, LAB3_REPORT.docx та LAB3_REPORT.pdf (усі скріншоти збережені в docs/screenshots/).

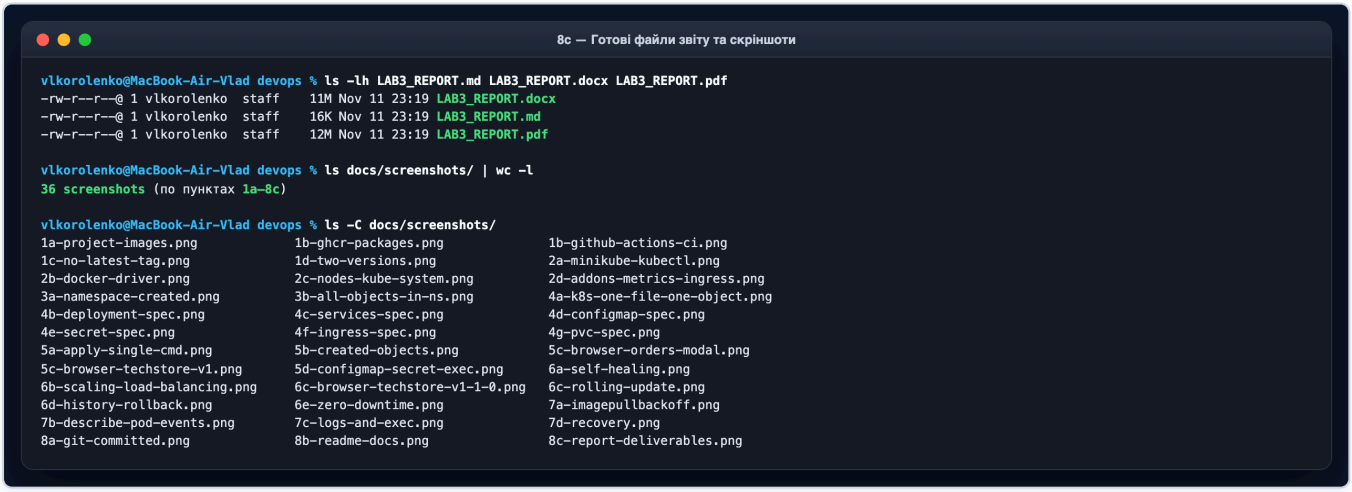


Рис. 8c — Готові файли звіту (MD, DOCX, PDF) та набір скріншотів у docs/screenshots/.

Висновки

У ході лабораторної роботи мікросервісний застосунок DevOps TechStore перенесено до Kubernetes у вигляді 13 ручних маніфестів у каталозі `k8s/` (окремий namespace `devops-techstore`, конкретні теги `1.0.0` і `1.1.0` без `latest`, постійний том PVC для PostgreSQL, параметри конфігурації через ConfigMap і Secret, доступ через NodePort та Ingress). Усі пункти завдання 1–8 перевірено й задокументовано окремо з доказами самовідновлення, масштабування, безперебійного Rolling Update, Rollback та діагностики ImagePullBackOff.